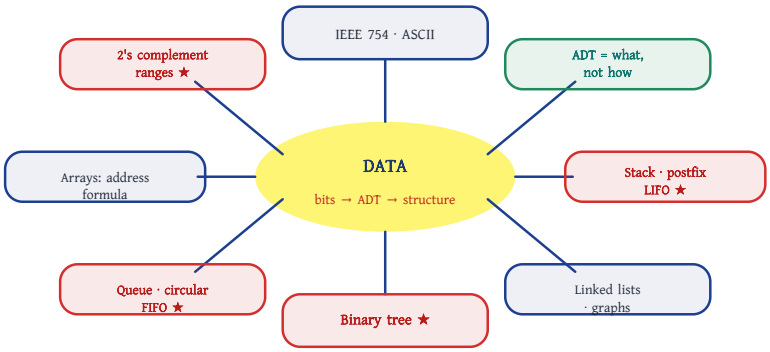
Data Representation & Abstract Data Types

bits · 2's complement · IEEE 754 · ADT · arrays · lists · stacks · queues · trees · graphs: exam-ready in 12 sheets



↑ the whole topic: red = high-yield, green = the one idea that ties it together

Syllabus coverage

The Data Structures and Algorithms syllabus opens: "**Internal data representation; Abstract data types; Elementary data structures: arrays, lists, stacks, queues, trees, graphs**; Advanced data structures: heaps, Fibonacci heaps, B-trees; Recursion, sorting, searching, hashing, storage management ..." **This lesson takes the first three clauses.** Heaps and B-trees, sorting/searching/hashing and graph algorithms have their own lessons.

High-yield list

① n-bit 2's complement: -2^{n-1} ... $2^{n-1}-1$

② −5 (8-bit 2's) = **11111011**

③ IEEE 754 single: 1 + 8 + 23, **bias 127**

④ ADT = operations, not implementation

⑤ Row-major: $B + w[(i-L_r)n + (j-L_c)]$

⑥ Stack = **LIFO**, queue = **FIFO**

⑦ Circular queue full: **(rear+1) % n == front**

⑧ 2 3 4 * + = **14**

⑨ Binary tree: $n_0 = n_2 + 1$, max $2^{h+1}-1$ nodes

⑩ Adj. matrix **$O(V^2)$** · list O(V+E)

Legend: **yellow** = key term · **red circle** = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

Contents

- ✓ Bits, types, signed integers ... p. 2
- ✓ IEEE 754, character codes ... p. 3
- ✓ Abstract data types ... p. 4
- ✓ Arrays & address formulas ... p. 5
- ✓ Linked lists ... p. 6
- ✓ Stacks, infix → postfix ... p. 7
- ✓ Queues, circular, deque ... p. 8
- ✓ Trees, binary trees ... p. 9
- ✓ Graphs, complexity table ... p. 10
- ✓ Night-before sheet ... p. 11
- ✓ Practice MCQs + written ... p. 12

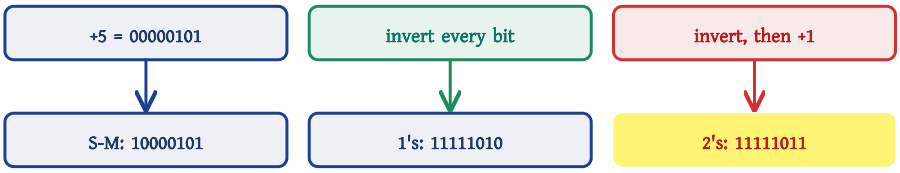
1 Bits, Data Types and Signed Integers

Definition: **Internal data representation** is how numbers, characters and instructions are encoded as **binary patterns**. **Bit** = one 0/1, **nibble** = 4 bits, **byte** = 8 bits, **word** = the CPU's natural unit (16/32/64 bits). The **type** gives a pattern its meaning: `01000001` is 65 as an integer, 'A' as ASCII. n bits give 2^n **patterns**, so **unsigned** n-bit integers run **0 … 2^n − 1** (8-bit 0…255, 16-bit 0…65535).

Primitive vs non-primitive: **Primitive**: built into the language — int, char, float, double, boolean, pointer. **Non-primitive**: built from them — **linear** (array, list, stack, queue) and **non-linear** (tree, graph). Base conversion is in the number-systems lessons; 1 hex digit = 4 bits.

C type	Size	Typical range
char / unsigned char	1 B	−128…127 / 0…255
short	2 B	−32768…32767
int / long long	4 / 8 B	$-2^{31}…2^{31}-1$ / -2^{63}…
float / double	4 / 8 B	~7 / ~15–16 digits

Three ways to store a negative number in n bits: **Sign-magnitude**: MSB = sign (0 +, 1 −), the rest = |x|. **1's complement**: invert every bit of +x. **2's complement**: invert every bit, then **add 1** (= 2^n − x). CPUs use **2's complement**: one zero, and one adder for both + and −.



−5 in 8 bits, three ways (4-bit: 1101 · 1010 · 1011)

Method	n-bit range	8-bit range	Zeros	−5 (4-bit)
Sign-magnitude	$-(2^{n-1}-1)$ … $2^{n-1}-1$	−127 … 127	**two** (+0, −0)	1101
1's complement	$-(2^{n-1}-1)$ … $2^{n-1}-1$	−127 … 127	**two**	1010
2's complement	-2^{n-1} … $2^{n-1}-1$	**−128 … 127**	one	**1011**

Memory trick: **"Copy up to the first 1, flip the rest."** Scan +x from the right, copy bits up to and including the first 1, invert the rest: 00000101 → 11111011. To **decode**, the MSB weighs -2^{n-1}: 11111011 = −128 + 123 = **−5**.

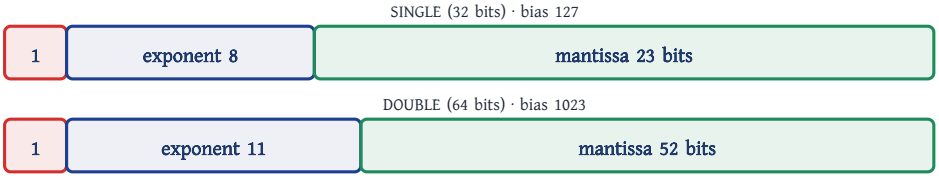
Exam trap: 2's complement is **asymmetric**: −128 exists in 8 bits, +128 does not. **10000101** (S-M) and **11111010** (1's) are the distractors for "2's complement of −5". To widen, **copy the sign bit**: 1011 → 1111 1011 (sign extension).

Overflow: Possible only when **two numbers of the same sign are added** and the result has the **opposite sign** (carry into MSB ≠ carry out). Opposite signs **never** overflow; a carry out of the MSB alone is simply discarded.

Worked example ✍ 8-bit: 100 + 50: `01100100 + 00110010 = 10010110` → MSB 1 from two positives: **overflow** (reads −106). 7 − 5 = 7 + (−5): `00000111 + 11111011 = 1 00000010` → drop the carry → **2**, no overflow.

2 IEEE 754 Floating Point and Character Codes

Definition: A real number is stored in normalised form $(-1)^S \times 1.M \times 2^{E - bias}$ with three fields: **sign** S, **biased exponent** E and **mantissa** (fraction) M. The leading 1 is **not stored** (hidden bit).



S-E-M: 1 + 8 + 23 = 32 · 1 + 11 + 52 = 64

Worked example ✍ −6.75 → single: **1.** Negative → S = **1**. **2.** 6.75 = **110.11**$_2$. **3.** Normalise: **1.1011 × 2^2**; drop the hidden 1, M = 1011 0…0. **4.** E = 2 + 127 = 129 = **10000001**. **5.** `1 10000001 1011000…0` = **C0D80000**h. **Check:** 1.1011_2 = 1.6875; × 2^2 = 6.75 ✓

Format	S	E	M	Bias	Digits
Single (32)	1	8	23	**127**	~7
Double (64)	1	11	52	**1023**	~15–16
E all 0s: ±0 (M = 0) or denormal · E all 1s: ±∞ (M = 0) or NaN					

Memory trick: **"1-8-23, 127; 1-11-52, 1023."** The bias is always $2^{k-1} - 1$ for a k-bit exponent: $2^7 - 1 = 127$, $2^{10} - 1 = 1023$. Field order **S-E-M**: Sign, Exponent, Mantissa.

Exam trap: **0.1 has no exact binary form** (0.000110011… recurring), so `0.1 + 0.2 == 0.3` is **false** in C, Java and Python. Compare floats with `|a − b| < ε`; keep money in integers (paisa).

Code	Size	Facts examiners use
BCD (8421)	4 bits per decimal digit	59 → `0101 1001`; 1010–1111 invalid; pure binary 59 = `111011`
ASCII	**7 bits**, 128 chars	'A' = 65 (41h), 'a' = 97 (61h), '0' = 48 (30h); cases differ by 32
Ext. ASCII / EBCDIC	8 bits	256 chars / IBM mainframes
Unicode	a character *set*	U+0000 … U+10FFFF; Bangla U+0980–U+09FF; 'অ' = U+0985
UTF-8 · 16 · 32	**1–4** · 2/4 · 4 bytes	UTF-8 is ASCII-compatible; 'অ' = `E0 A6 85`

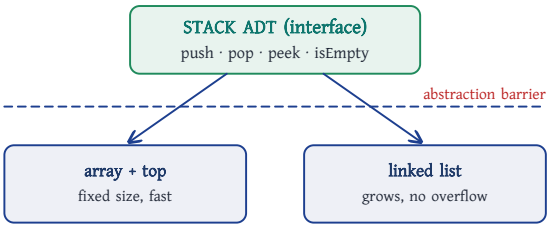
Worked example ✍ endianness: `0x12345678` at address 100. **Big-endian**: 100→12, 101→34, 102→56, 103→78. **Little-endian**: 100→**78** … 103→12. x86 is little-endian; **network byte order is big-endian**.

Exam trap: "Unicode is 16-bit" is outdated: Unicode is a set of 1 114 112 code points and **UTF-8/16/32 are its encodings**. ASCII is **7-bit**, not 8. `int`'s size is **compiler-dependent** (≥ 16 bits in C).

3 Abstract Data Types

Definition: An **abstract data type (ADT)** is a mathematical model of a type defined by **its values and the operations on them** (with their rules), **not by how it is implemented.** A **data structure** is a concrete way of arranging data in memory that implements an ADT. "Stack" is an ADT; "array with a `top` index" is one data structure that implements it.

> what, not how



one interface, many implementations

Point	ADT	Data structure
Level	logical, abstract	physical, concrete
Says	**what** it does	**how** it does it
Contains	values + operations + rules	memory layout + algorithms
Example	Stack, Queue, List, Map	array, linked list, heap, hash table
In code	interface / header	class / .c file

ADT	Core operations	Rule	Built with
List	insert, delete, get(i), search	position	array, linked list
Stack	push, pop, peek, isEmpty	**LIFO**	array, linked list
Queue	enqueue, dequeue, front, rear	**FIFO**	circular array, list
Deque	insert/delete at both ends	both ends	doubly linked list
Priority queue	insert, extract-max, peek	priority	**binary heap**
Set / Map	add, contains · put(k,v), get(k)	no duplicates	hash table, BST
Tree	insert, delete, search, traverse	one parent	linked nodes, array
Graph	addVertex, addEdge, adjacent	any to any	adj. matrix / list

Why ADTs — three benefits: **Encapsulation** (information hiding): users see only the operations, so the representation can change without breaking them. **Reusability**: the same ADT can be implemented several ways and reused in many programs. **Modularity**: each module is written, tested and debugged on its own. Together they give **data abstraction**, the basis of classes in OOP.

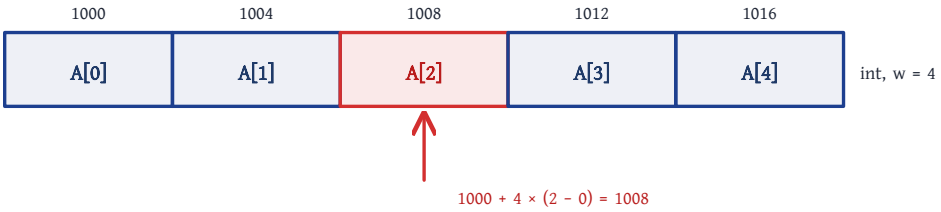
Memory trick: **ADT = a menu; data structure = the kitchen.** The menu lists what you can order (push, pop); the kitchen decides how it is cooked (array or list). Change the kitchen, keep the menu. Benefits: "**E-R-M**" — Encapsulation, Reusability, Modularity.

Exam trap: BFS and DFS are **algorithms** run on a graph, not operations of the Graph ADT itself. A priority queue is **not FIFO**: it serves by priority. A linked list is a **data structure**; "List" is the ADT. Asked "an ADT defines...?" → **behaviour**, not storage.

4 Arrays and Address Calculation

Definition: An **array** is a fixed-size collection of elements of **one type in contiguous memory**, reached by index. Because the elements are contiguous and equally sized, the address of any element is computed by a formula, so **access is O(1)** (random access). Insertion or deletion in the middle shifts elements: **O(n)**.

random access O(1)



base address + width × offset

Address formulas: B = base address, w = bytes per element, L = lower bound.

1-D: LOC(A[i]) = **B + w (i − L)** · number of elements = U − L + 1.

2-D, m rows × n columns, row-major (C, C++, Java, Python): LOC(A[i][j]) = **B + w [(i − L_r) · n + (j − L_c)]**.

2-D, column-major (Fortran, MATLAB): LOC(A[i][j]) = **B + w [(j − L_c) · m + (i − L_r)]**.

Row-major multiplies by the **number of columns n**; column-major by the **number of rows m**.

Worked example ✍ 2-D, both orders: `int A[20][30]` (0-based, m = 20, n = 30), B = 1000, w = 4. Find A[5][10].

Row-major: 1000 + 4 (5 × 30 + 10) = 1000 + 4 × 160 = **1640**.

Column-major: 1000 + 4 (10 × 20 + 5) = 1000 + 4 × 205 = **1820**.

Worked example ✍ non-zero lower bound: `A[5..50]`, B = 200, w = 2. Elements = 50 − 5 + 1 = **46**. LOC(A[20]) = 200 + 2 (20 − 5) = **230**.

`A[-5..10]` holds 10 − (−5) + 1 = **16** elements.

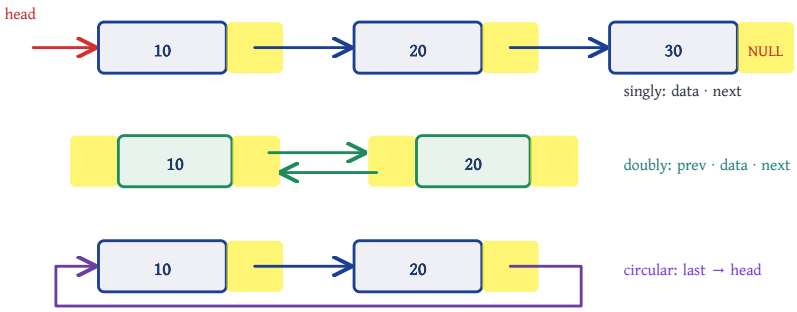
Operation on an array of n	Time	Why
Access A[i]	**O(1)**	address by formula
Search (unsorted / sorted)	O(n) / O(log n)	linear scan / binary search
Insert / delete at end	O(1)	no shifting (if space remains)
Insert / delete at position k	**O(n)**	shift n − k elements

Sparse matrix: Most elements are 0. Store only the non-zeros as **triplets (row, col, value)**, or as a linked list. A 100 × 100 matrix with 50 non-zeros needs 3 × 50 = 150 cells instead of 10 000. An n × n **lower-triangular** matrix has **n(n+1)/2** stored elements.

Exam trap: Check the **lower bound**: with 1-based `A[1..m][1..n]`, subtract 1 from i and j first. Read whether the question says row- or column-major. C has no bounds checking: `A[10]` in `int A[10]` is **undefined behaviour**, not an error message.

5 Linked Lists: Singly, Doubly, Circular

Definition: A **linked list** is a sequence of **nodes**, each holding data and a **pointer** to the next node; the nodes may lie anywhere in memory. A **head** pointer reaches the first node and the last node's `next` is NULL. Size grows and shrinks at run time; there is no random access, so reaching the k-th node is **O(n)**.



singly · doubly · circular

```
struct Node { int data;
  struct Node *next; };

/* insert at head: O(1) */
struct Node *n = malloc(sizeof *n);
n->data = x;
n->next = head;
head = n;

/* delete the node after p: O(1) */
struct Node *t = p->next;
p->next = t->next;
free(t);
```

Type	Node	Use
Singly	data, next	stacks, simple lists
Doubly	prev, data, next	browser back/forward, LRU cache, deque
Circular	last → first	**round-robin scheduling**
Circular doubly	both links wrap	music playlists, Fibonacci heap roots
Header list	dummy first node	no special case for empty list

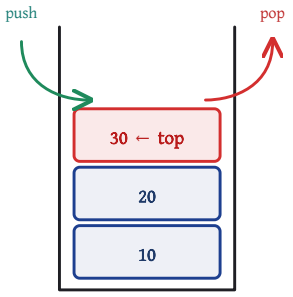
Point	Array	Linked list
Memory	contiguous, fixed size	scattered, grows at run time
Access k-th	**O(1)**	**O(n)**
Insert/delete at front	O(n) (shift)	**O(1)**
Insert at end	O(1) if space	O(n), or O(1) with a tail pointer
Extra space	none (may waste unused slots)	one pointer per node (two if doubly)

Memory trick: **"Link first, then cut."** When inserting, set the new node's `next` *before* redirecting the old pointer, or the rest of the list is lost. Middle node: **slow/fast pointers** (fast moves 2, slow 1). Cycle: Floyd's tortoise and hare.

Exam trap: Deleting a given node in a **singly** list needs its **predecessor**, so it costs O(n) to find it; in a **doubly** list it is **O(1)**. Reversing a singly list takes O(n) time and O(1) space (three pointers: prev, curr, next).

6 Stacks, Infix → Postfix and Postfix Evaluation

Definition: A **stack** is a list in which insertion (**push**) and deletion (**pop**) happen at **one end, the top**: **Last In, First Out**. Array version: `top = -1` means empty; push when `top == MAX-1` is **overflow**; pop when `top == -1` is **underflow**. Linked version pushes and pops at the head and never overflows (until memory runs out). All operations **O(1)**.



last in, first out

```
void push(int x) {
  if (top == MAX-1) { puts("Overflow"); return; }
  s[++top] = x; }
int pop(void) {
  if (top == -1) { puts("Underflow"); return -1; }
  return s[top--]; }
```

Applications: **function calls and recursion** (the call stack holds return addresses and locals) · expression conversion and evaluation · balanced-parenthesis check · undo/redo · browser back · backtracking (maze, N-queens) · DFS · reversing a string. Every recursive algorithm can be rewritten with an explicit stack.

Infix → postfix rules: Scan left to right. **Operand** → output. **(** → push. **)** → pop to output until "(". **Operator** → pop while the top has **higher or equal** precedence (equal only if left-associative), then push. At the end pop everything. ^ is **right**-associative. Precedence: **^ > * / > + -**

Read	Stack	Output
A		A
+	+	A
B	+	A B
*	+ *	A B
C	+ *	A B C
-	-	A B C * +
D, /, E	- /	A B C * + D E
end		**A B C * + D E / -**

Worked example ✍ evaluate postfix: Operand → push; operator → pop **B** (top), pop **A**, push **A op B**.
`2 3 4 * +`: push 2, 3, 4; * → 3 × 4 = 12; + → 2 + 12 = **14** (infix 2 + 3 * 4).
`5 6 2 + * 12 4 / -`: 6 + 2 = 8; 5 × 8 = 40; 12 / 4 = 3; 40 - 3 = **37**.

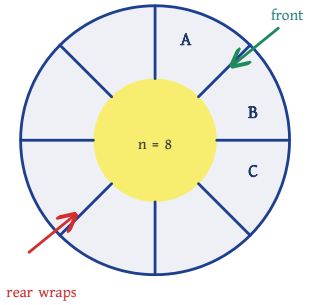
Worked example ✍ three notations: Infix `(A + B) * (C - D)`
Postfix (reverse Polish): **A B + C D - ***
Prefix (Polish): *** + A B - C D**
Postfix needs **no parentheses and no precedence rules**, which is why compilers and calculators use it.

Memory trick: **"Stack of plates."** The last plate washed is the first one used. For postfix evaluation, the **second pop is the left operand**: "pop B, pop A, compute A op B" — matters for - and /.

Exam trap: `2 3 4 * +` is **14, not 20**; 20 would be `2 3 + 4 *`. Pushing 1, 2, 3 in order can never pop as **3, 1, 2**. Recursion that never reaches its base case ends in **stack overflow**.

7 Queues: Linear, Circular, Deque, Priority

Definition: A **queue** inserts at the **rear** (*enqueue*) and deletes from the **front** (*dequeue*): **First In, First Out.** In a plain array front and rear only move forward, so freed slots are never reused ("full" while half empty); a **circular queue** wraps the indices with `% n`.



rear = (rear + 1) % n

Circular queue (array of n)	Condition
Enqueue	`rear = (rear + 1) % n; Q[rear] = x`
Dequeue	`x = Q[front]; front = (front + 1) % n`
Full	**(rear + 1) % n == front**
Empty	`front == -1` (or `front == rear` in the one-slot-free form)
Elements	`(rear - front + n) % n` (one-slot-free form)

Two conventions: `front = rear = -1` as empty: all n slots usable. `front = rear = 0` as empty: one slot stays free, capacity **`n - 1`**.

Worked example ✍️ trace: Empty queue: enqueue(1), enqueue(2), enqueue(3) → [1, 2, 3]; dequeue() removes **1** → [2, 3]; enqueue(4) → [2, 3, 4]; dequeue() removes **2** → [3, 4]. Front = **3**, rear = 4. The same pushes and pops on a **stack** leave [1, 2] with 2 on top.

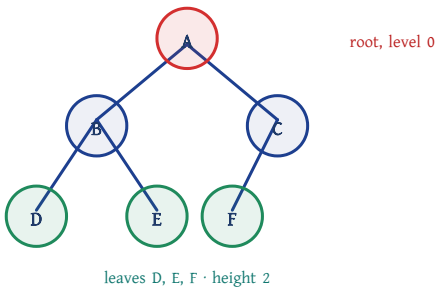
Variant	Rule	Typical use
Linear queue	FIFO, indices only move forward	simple buffers
Circular queue	FIFO, indices wrap with % n	CPU **round-robin**, keyboard and network buffers
Deque	insert and delete at **both** ends	sliding-window maximum, undo history
Restricted deque	input-restricted: insert at one end · output-restricted: delete at one end	—
Priority queue	highest priority served first; ties FIFO	Dijkstra, OS job scheduling, Huffman

Memory trick: **"Bank counter queue."** Join at the back (rear), leave from the front. **D**ouble-**E**nded **QUE**ue = deque. Uses: BFS, printer spooling, CPU scheduling, buffers between fast and slow devices.

Exam trap: A priority queue is best built on a **binary heap**: insert and delete-max both O(log n) — an unsorted array makes delete O(n). A queue can be built from **two stacks** (amortised O(1)). "Which structure for BFS?" → queue; "for DFS?" → stack.

8 Trees and Binary Trees

Definition: A **tree** is a connected acyclic hierarchy with one **root**; every other node has exactly one parent, so n nodes have **n − 1 edges**. A **binary tree**: at most two children (left, right). Terms: **leaf**, **internal** node, **siblings**, **degree** = number of children, **level** (root = 0), **height** = edges on the longest root-to-leaf path.



F is C's **left** child

Traversal	Order	Result for this tree
Preorder	Node, Left, Right	A B D E C F
Inorder	Left, Node, Right	D B E A F C
Postorder	Left, Right, Node	**D E B F C A**
Level order	level by level (queue)	A B C D E F

Binary-tree property (root at level 0, height h)	Formula	Check (h = 2)
Max nodes at level i	2^i	level 2: 4
Max nodes in a tree of height h	$2^{h+1} - 1$	7
Min height with n nodes	$\lceil \log_2(n + 1) \rceil - 1$	n = 7 → 2
Leaves vs nodes of degree 2	$n_0 = n_2 + 1$	perfect h = 2: 4 = 3 + 1
Distinct binary trees with n nodes	Catalan: C(2n, n)/(n + 1)	n = 3 → 5

Kinds of binary tree: **Full / strict**: every node has 0 or 2 children. **Complete**: all levels full except the last, filled **from the left** (heap shape). **Perfect**: every internal node has 2 children, all leaves on one level. **Skewed**: one child each — a list.

Array representation: Store level by level. **0-based:** left = **2i + 1**, right = 2i + 2, parent = ⌊(i − 1)/2⌋. **1-based:** left = 2i, right = 2i + 1, parent = ⌊i/2⌋. Ideal for complete trees; wasteful for skewed ones.

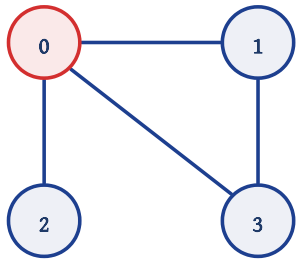
Worked example ✍️ rebuild from two traversals: Preorder A B D E C F, inorder D B E A F C. Preorder's first is the root **A**; in the inorder, D B E is its left subtree and F C its right. **B** roots the left part (D, E); **C** roots the right with F on its left. Postorder: **D E B F C A**. Inorder + pre/post fixes a tree uniquely; pre + post alone does not.

Memory trick: The name says **where the Node goes**: **Pre** = node first, **In** = node in the middle, **Post** = node last; Left always comes before Right. **BST**: left < node < right, so its **inorder is sorted**.

Exam trap: Check where the level count starts: with root at level **1**, max nodes at level i = 2^{i-1} and in height h = $2^h - 1$. BST search is O(log n) **on average** but **O(n) when skewed** — hence AVL and red-black trees.

9 Graphs, and the Cost of Every Operation

Definition: A **graph** G = (V, E) is a set of vertices and a set of edges joining pairs of them. **Undirected** or **directed** (digraph), **weighted** or not. **Degree** = edges at a vertex (in-/out-degree in a digraph). A **path** joins vertices by edges; a **cycle** returns to its start; a graph is **connected** if every pair is joined by a path. A **tree is a connected acyclic graph**.



V = 4, E = 4

undirected, 4 edges

·	0	1	2	3
0	0	1	1	1
1	1	0	0	1
2	1	0	0	0
3	1	1	0	0

```
0 → 1 → 2 → 3
1 → 0 → 3
2 → 0
3 → 0 → 1
```

Point	Adjacency matrix	Adjacency list
Space	**O(V²)**	**O(V + E)**
Is (u, v) an edge?	**O(1)**	O(degree of u)
List all neighbours of u	O(V)	O(degree of u)
Best for	**dense** graphs	**sparse** graphs (most real ones)

Counting facts: Max edges: undirected simple graph **n(n − 1)/2**, directed n(n − 1) — so K_5 has 10 edges.

Handshaking: sum of all degrees = 2|E|, so the number of odd-degree vertices is even. A tree on n vertices has n − 1 edges; a **DAG** is a directed graph with no cycle. An **incidence matrix** is V × E.

Structure	Access	Search	Insert	Delete
Array (unsorted)	O(1)	O(n)	O(n)	O(n)
Sorted array	O(1)	O(log n)	O(n)	O(n)
Singly linked list	O(n)	O(n)	O(1) at head	O(1) after a known node
Stack / Queue	top/front O(1)	O(n)	O(1)	O(1)
BST (average / worst)	—	O(log n) / O(n)	O(log n) / O(n)	O(log n) / O(n)
Binary heap	max O(1)	O(n)	O(log n)	O(log n)

Exam trap: "Space of an adjacency list" is **O(V + E)**, not O(E): every vertex needs a list head even with no edges. A complete graph is dense, so a matrix wins there. A graph may have cycles and several components; a tree may not.

Where the rest of this chapter lives: **Heaps, Fibonacci heaps, B-trees** → the advanced data structures lesson. **Recursion, sorting, searching, hashing** → the next one. **BFS, DFS, topological sort, MST, shortest paths** → the graph algorithms lesson.

Night-Before-the-Exam Sheet

Fact sheet:

n bits → 2^n patterns; unsigned 0…2^n−1

2's comp: -2^{n-1} … 2^{n-1}−1 (8-bit −128…127)

−5: S-M 10000101 · 1's 11111010

−5 in 2's (8-bit) = **11111011**

Overflow: same signs in, opposite sign out

ASCII 7-bit · 'A' 65 · 'a' 97 · '0' 48

IEEE single 1-8-23, bias 127

IEEE double 1-11-52, bias 1023

ADT = what (operations); DS = how

Little-endian: low byte at low address

Row-major: B + w[(i−L_r)n + (j−L_c)]

Col-major: B + w[(j−L_c)m + (i−L_r)]

Stack LIFO · overflow top == MAX−1

Circular full: (rear+1) % n == front

Level i ≤ 2^i nodes; height h ≤ 2^{h+1}−1

$n_0 = n_2 + 1$ · edges = n − 1

Children 2i+1, 2i+2 (0-based)

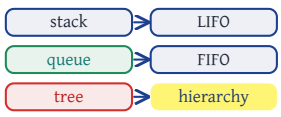
Matrix $O(V^2)$ · list O(V+E) · max edges n(n−1)/2

Top 12 one-liners for MCQs

1. 2's complement has one zero; S-M and 1's have two
2. Copy to the first 1, flip the rest
3. Unicode is a set; UTF-8 is 1–4 bytes
4. 0.1 + 0.2 ≠ 0.3 in floating point
5. Stack ADT: push, pop, peek, isEmpty
6. Array access O(1); insert in middle O(n)
7. Doubly list deletes a given node in O(1)
8. 2 3 4 * + = 14; A+B*C → ABC*+
9. Precedence ^ > * / > + −; ^ right-assoc.
10. Recursion uses the stack; BFS uses a queue
11. Priority queue → binary heap
12. Inorder of a BST is sorted

Top traps: 10000101 is S-M, not 2's · −128 has no +128 partner · carry out ≠ overflow · ASCII is 7-bit · int size is compiler-dependent · lower bound ≠ 0 · second pop is the left operand · priority queue is not FIFO · BST worst case O(n) · level count from 0 or 1.

All tricks together: Copy to the first 1, flip the rest · 1-8-23/127, 1-11-52/1023, bias 2^{k-1}−1 · ADT = menu, DS = kitchen · link first, then cut · stack of plates · bank counter queue · Pre/In/Post = where the Node goes.



three orders

Job	Structure	Job	Structure
recursion, undo, postfix	**stack**	BFS, printer spool	**queue**
round-robin CPU	circular queue / list	job priority, Dijkstra	priority queue (heap)

Worked example ✍ five numbers in one minute: (1) 8-bit 2's of −20: 00010100 → **11101100**. (2) 16-bit 2's range: **−32768 … 32767**. (3) `A[10][10]`, B = 500, w = 2, row-major A[3][4]: 500 + 2 (34) = **568**. (4) Binary tree with 10 leaves has **9** nodes of degree 2. (5) Simple undirected graph on 6 vertices: at most **15** edges.

✓ Practice MCQs (exam pattern, 10 marks)

1. Which ADT follows LIFO?
(a) Queue (b) Stack
(c) Priority queue (d) Circular queue

2. 8-bit two's complement of −5:
(a) 10000101 (b) 11111010
(c) 11111011 (d) 00000101

3. Range of 8-bit two's complement:
(a) −127 … 127 (b) −128 … 127
(c) 0 … 255 (d) −128 … 128

4. Exponent bias of IEEE 754 single:
(a) 127 (b) 128
(c) 1023 (d) 255

5. `int A[20][30]`, B = 1000, w = 4, row-major A[5][10]:
(a) 1820 (b) 1640
(c) 1600 (d) 1660

6. Value of postfix `2 3 4 * +`:
(a) 20 (b) 24
(c) 9 (d) 14

7. A circular queue of size n is full when:
(a) front == rear (b) rear == n − 1
(c) (rear+1)%n == front (d) front == 0

8. enq 1, 2, 3, deq, enq 4, deq. Front?
(a) 3 (b) 4
(c) 2 (d) 1

9. Max nodes in a binary tree of height 3:
(a) 8 (b) 7
(c) 15 (d) 16

10. Space of an adjacency matrix:
(a) O(V + E) (b) O(E)
(c) O(V log V) (d) $O(V^2)$

Answer key → 1-b · 2-c · 3-b · 4-a · 5-b · 6-d · 7-c · 8-a · 9-c · 10-d

Written question: What is an abstract data type? Describe the Stack ADT and its array implementation with overflow and underflow. Convert `A + B * C - D / E` to postfix showing the stack, and evaluate `5 6 2 + * 12 4 / -`. 10 marks

1. **ADT (2)** — a type defined by its values and operations, independent of implementation; benefits: encapsulation, reusability, modularity. A data structure implements it. 2
2. **Stack ADT (2)** — LIFO list; push, pop, peek, isEmpty, isFull, all O(1). Uses: call stack, expressions, undo, backtracking. 2
3. **Array implementation (2)** — `top = -1` empty; push: if `top == MAX-1` → **overflow**, else `s[++top] = x`; pop: if `top == -1` → **underflow**, else return `s[top--]`. 2
4. **Infix → postfix (2)** — A out; push +; B out; * outranks +, push; C out; − pops * and + (≥ precedence), push −; D out; push /; E out; pop / and − → **A B C * + D E / −**. 2
5. **Evaluate (2)** — push 5, 6, 2; + → 8; * → 40; push 12, 4; / → 3; − → 40 − 3 = **37**. 2

Before the exam ✓

- ✓ Can write −x in S-M, 1's and 2's complement
- ✓ Can lay out IEEE 754 single and double
- ✓ Can define an ADT and contrast it with a DS
- ✓ Can apply both 2-D address formulas
- ✓ Can convert infix → postfix and evaluate it
- ✓ Can state the circular-queue and tree formulas